



**Министерство науки и высшего образования Российской
Федерации**
**Федеральное государственное автономное образовательное
учреждение высшего образования**
**«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

**Лабораторная работа № 7
по дисциплине «Операционные системы»**

Тема Буферизованный и не буферизованный ввод-вывод

Студент Павлов Д.В.

Группа ИУ7-63Б

Преподаватель Рязанова Н.Ю.

Москва, 2026

1 Структуры

Структура stat

```
struct stat {
    dev_t st_dev;    /* inode's device */
    ino_t st_ino;    /* inode's number */
    mode_t st_mode;  /* inode protection mode */
    nlink_t st_nlink; /* number of hard links */
    uid_t st_uid;    /* user ID of the file's owner */
    gid_t st_gid;    /* group ID of the file's group */
    dev_t st_rdev;   /* device type */
    struct timespec st_atime; /* time of last access */
    struct timespec st_mtime; /* time of last data modification */
    struct timespec st_ctime; /* time of last file status change */
    off_t st_size;   /* file size, in bytes */
    blkcnt_t st_blocks; /* blocks allocated for file */
    blksize_t st_blksize; /* optimal blocksize for I/O */
    u_int32_t st_flags; /* user defined flags for file */
    u_int32_t st_gen;   /* file generation number */
};
```

Структура _IO_FILE

```
struct _IO_FILE {
    int _flags;          /* High-order word is _IO_MAGIC; rest is flags. */
    #define _IO_file_flags _flags

    /* The following pointers correspond to the C++ streambuf protocol. */
    char* _IO_read_ptr;  /* Current read pointer */
    char* _IO_read_end;  /* End of get area. */
    char* _IO_read_base; /* Start of putback+get area. */
    char* _IO_write_base; /* Start of put area. */
    char* _IO_write_ptr; /* Current put pointer. */
    char* _IO_write_end; /* End of put area. */
    char* _IO_buf_base;  /* Start of reserve area. */
    char* _IO_buf_end;   /* End of reserve area. */
    /* The following fields are used to support backing up and undo. */
    char *_IO_save_base; /* Pointer to start of non-current get area. */
    char *_IO_backup_base; /* Pointer to first valid character of backup
        area */
    char *_IO_save_end; /* Pointer to end of non-current get area. */

    struct _IO_marker *_markers;
```

```

struct _IO_FILE *_chain;

int _fileno;
#if 0
int _blksize;
#else
int _flags2;
#endif
_IO_off_t _old_offset; /* This used to be _offset but it's too small.
    */

#define __HAVE_COLUMN /* temporary */
/* 1+column number of pbase(); 0 is unknown. */
unsigned short _cur_column;
signed char _vtable_offset;
char _shortbuf[1];

/* char* _save_gptr; char* _save_egptr; */

_IO_lock_t *_lock;
#ifdef _IO_USE_OLD_IO_FILE
};

```

2 Практическая часть

Первая программа. Один поток

```
#include <stdio.h>
#include <fcntl.h>

int main()
{
    int fd = open("alphabet.txt", O_RDONLY);
    FILE *fs1 = fdopen(fd, "r");
    char buff1[20];
    setvbuf(fs1, buff1, _IOFBF, 20);
    FILE *fs2 = fdopen(fd, "r");
    char buff2[20];
    setvbuf(fs2, buff2, _IOFBF, 20);
    int flag1 = 1, flag2 = 2;
    while (flag1 == 1 || flag2 == 1)
    {
        char c;
        flag1 = fscanf(fs1, "%c", &c);
        if (flag1 == 1)
        {
            fprintf(stdout, "%c", c);
        }
        flag2 = fscanf(fs2, "%c", &c);
        if (flag2 == 1)
        {
            fprintf(stdout, "%c", c);
        }
    }
    printf("\n");
    return 0;
}
```

Данная программа считывает информацию из файла «alphabet.txt», который содержит строку символов «abcdefghijklmnopqrstuvwxyz». И при помощи двух буферов посимвольно выводит считанные символы в стандартный поток вывода stdout.

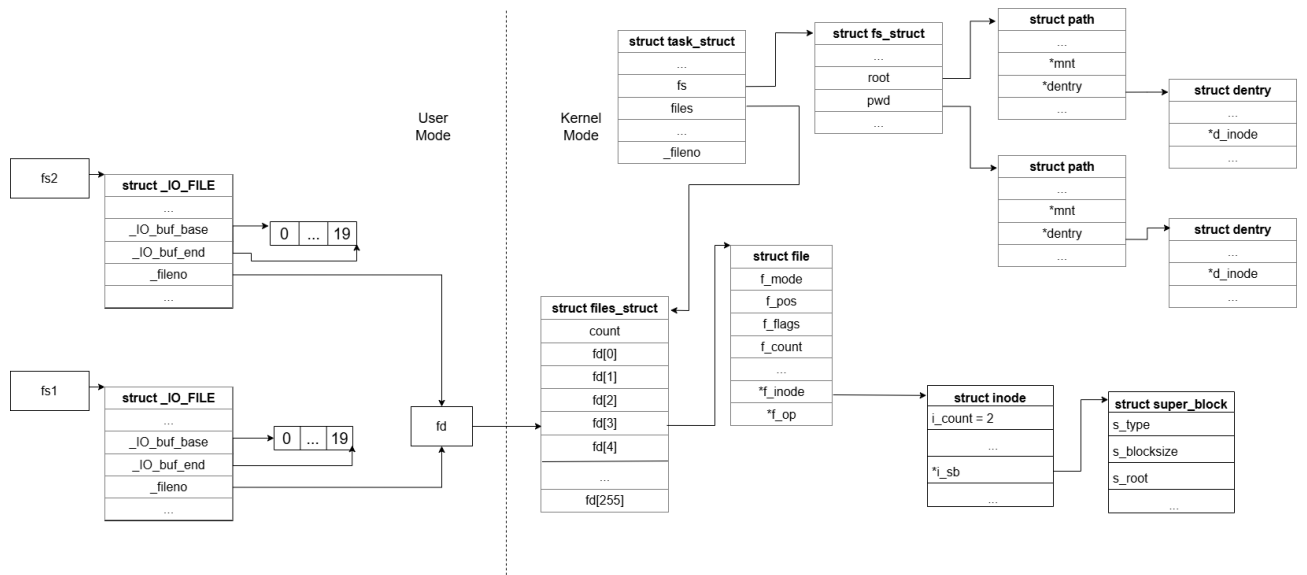


Рис. 2.1 — Связь структур в первой программе

В начале функции `main` `open()` создает новый файловый дескриптор для открытого только на чтение (`O_RDONLY`) файла «alphabet.txt», запись в системной таблице открытых файлов. Эта запись регистрирует смещение в файле и флаги состояния файла.

Далее `fdopen()` создает два указателя на структуру `FILE`, приведенную выше. В данных структурах поле `_fileno` будет содержать дескриптор, который вернула функция `fdopen()`. Для `fs1` и `fs2` эти поля будут равны 3.

Функция `setvbuf()` изменяет тип буферизации для `fs1` и `fs2` на полную буферизацию, а также явно задает размер буфера 20 байт.

Далее при первом вызове `fscanf()` буфер `fs1` заполнится полностью, т.е. первыми 20 символами. Значение `f_pos` в структуре `struct_file` открытого файла увеличится на 20. Далее при первом вызове `fscanf()` для `fs2` в `buff2` считаются оставшиеся 6 символов, начиная с `f_pos` (т.к. `fs1` и `fs2` ссылаются на один и тот же дескриптор `fd`).

Далее в цикле поочередно выводятся символы из `buff1` и `buff2`. Т.к. в `buff2` записались оставшиеся 6 символов, после 6 итерации цикла будут выводиться символы только из `buff1`.

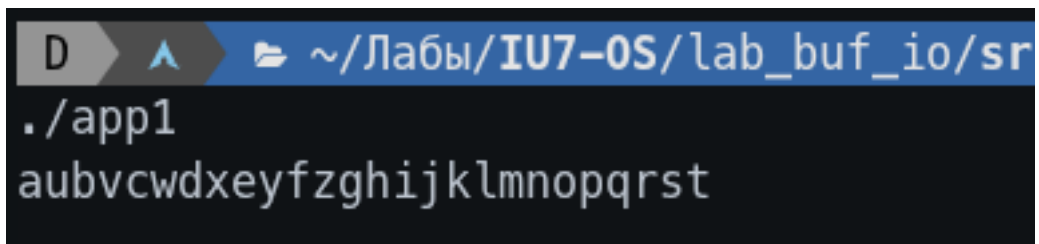


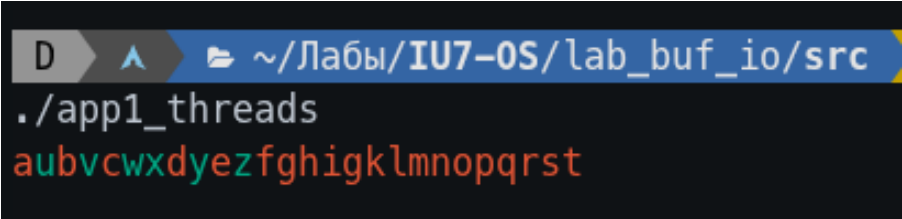
Рис. 2.2 — Результат работы первой программы

Первая программа. Два потока

```
#include <stdio.h>
#include <fcntl.h>
#include <pthread.h>
#include <unistd.h>
void *launchThread(void *args)
{
    int rc2 = 1;
    FILE *fs = (FILE *)args;
    while (rc2 == 1)
    {
        char c;
        if ((rc2 = fscanf(fs, "%c\n", &c)) == 1)
        {
            fprintf(stdout, "%c", c);
            sleep(1);
        }
    }
    return NULL;
}

int main(void)
{
    int fd = open("alphabet.txt", O_RDONLY);
    FILE *fs1 = fdopen(fd, "r");
    char buf1[20];
    setvbuf(fs1, buf1, _IOFBF, 20);
    FILE *fs2 = fdopen(fd, "r");
    char buf2[20];
    setvbuf(fs2, buf2, _IOFBF, 20);
    pthread_t thread;
    int rc = pthread_create(&thread, NULL, launchThread, (void *)fs2);
    int rc1 = 1;
    while (rc1 == 1)
    {
        char c;
        rc1 = fscanf(fs1, "%c\n", &c);
    }
}
```

```
        if (rc1 == 1)
        {
            fprintf(stdout, "%c", c);
            sleep(1);
        }
    }
    pthread_join(thread, NULL);
    printf("\n");
    return 0;
}
```

A terminal window with a dark background. The top bar shows the directory path ~/Лабы/IU7-OS/lab_buf_io/src. The command ./app1_threads has been executed, resulting in the output string 'aubvcwxdyezfg h i g k l m n o p q r s t' where the first 10 characters are in orange and the next 10 are in green, indicating interleaved output from two threads.

```
D  ^  ~/Лабы/IU7-OS/lab_buf_io/src
./app1_threads
aubvcwxdyezfg h i g k l m n o p q r s t
```

Рис. 2.3 — Результат работы первой программы с двумя потоками

Вторая программа. Один поток

```
#include <fcntl.h>
#include <unistd.h> // read, write.

int main()
{
    char c;
    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);
    int rc1, rc2 = 1;
    while (rc1 == 1 || rc2 == 1){
        char c;
        rc1 = read(fd1, &c, 1);
        if (rc1 == 1){
            write(1, &c, 1);}
        rc2 = read(fd2, &c, 1);
        if (rc2 == 1){
            write(1, &c, 1);}
    }
    write(1, "\n", 1);
    return 0;
}
```

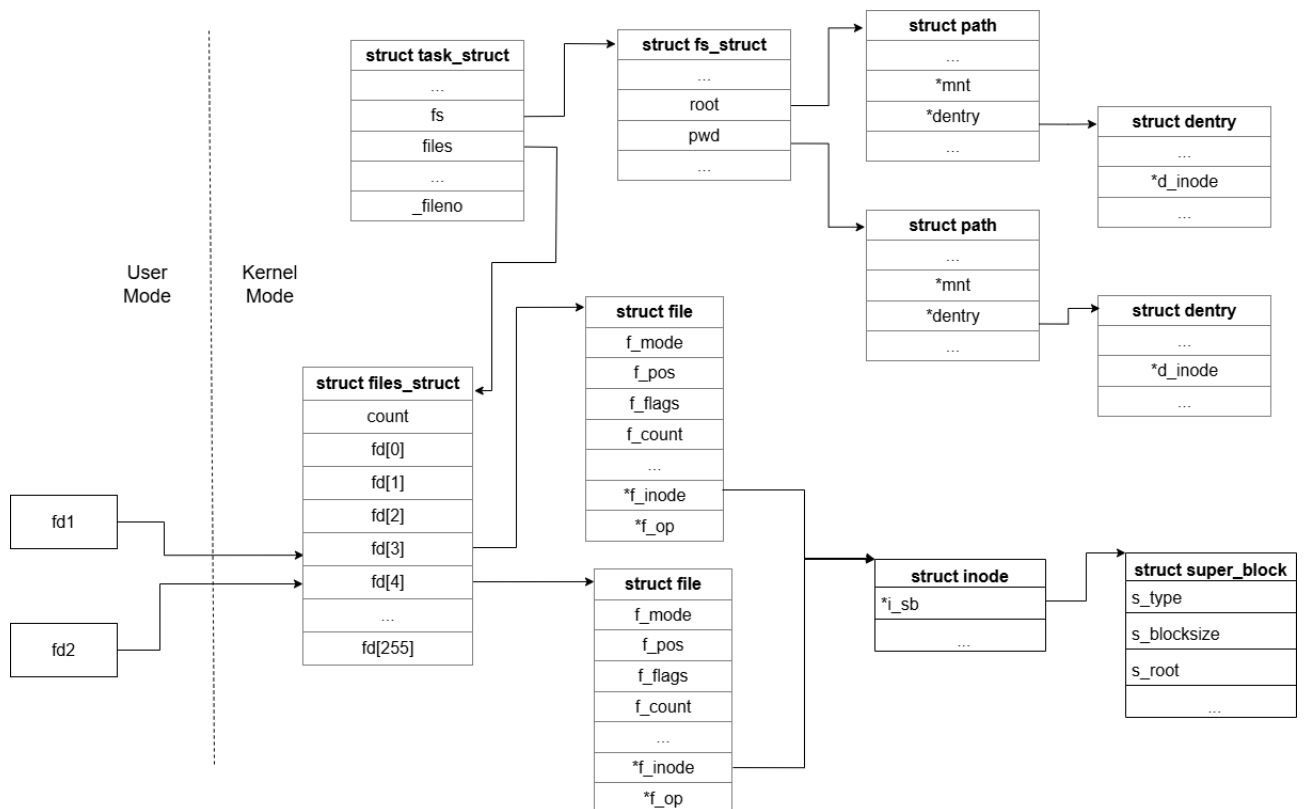


Рис. 2.4 — Связь структур во второй программе

В данной программе создается два дескриптора открытого файла при помощи функции `open()`. В системной таблице открытых файлов создаются две новых записи. Далее в цикле поочередно считываются символы из файла и выводятся на экран. Т.к. созданы две структуры `struct file`, то у каждой структуры будет свой `f_pos` и смещения в файловых дескрипторах будут независимы, поэтому на экран будут дважды выводиться символы одного и того же файла.

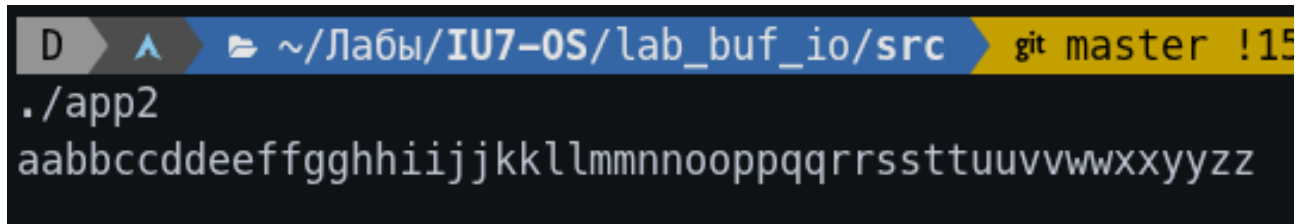


Рис. 2.5 — Результат работы второй программы

Вторая программа. Два потока

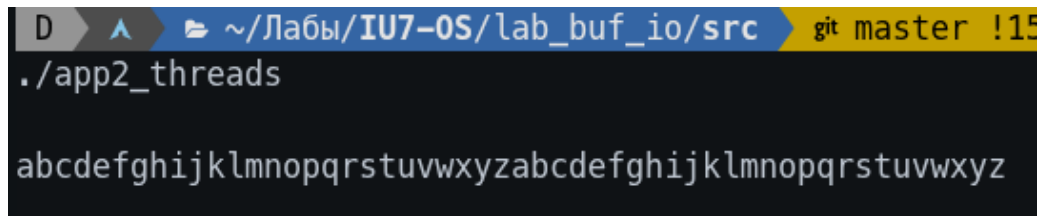
```
#include <fcntl.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void *reader(void *arg)
{
    int fd = open("alphabet.txt", O_RDONLY);
    char c;
    int flag = 1;
    while (flag == 1)
    {
        flag = read(fd, &c, 1);
        if (flag == 1)
        {
            write(1, &c, 1);
        }
    }
    close(fd);
    pthread_exit(NULL);
}

int main(void)
{
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, reader, NULL);
```

```
pthread_create(&thread2, NULL, reader, NULL);  
pthread_join(thread1, NULL);  
pthread_join(thread2, NULL);  
write(1, "\n", 1);  
return 0;  
}
```

В программе также, как и при реализации с одним потоком, создается два файловых дескриптора для открытого файла, записи в системной таблице открытых файлов. У каждой записи будет свое смещение `f_pos`. Т.к. главный поток ждет окончания дочернего, то гарантируется вывод всего алфавита дважды. В многопоточной реализации каждый символ будет выведен дважды, но последовательность символов не определена.



```
D ~ /Лабы/IU7-0S/lab_buf_io/src git master !15  
./app2_threads  
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz
```

Рис. 2.6 — Результат работы второй программы с двумя потоками

Третья программа

Небуферизованный ввод

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

int main(void)
{
    int fd1 = open("3.txt", O_RDWR);
    int fd2 = open("3.txt", O_RDWR);
    print_stat();
    int curr = 0;
    for (char c = 'a'; c <= 'z'; c++)
    {
        if (c % 2)
        {
            write(fd1, &c, 1);
            print_stat();
        }
        else
        {
            write(fd2, &c, 1);
            print_stat();
        }
    }
    close(fd1);
    print_stat();
    close(fd2);
    print_stat();
    return 0;
}
```

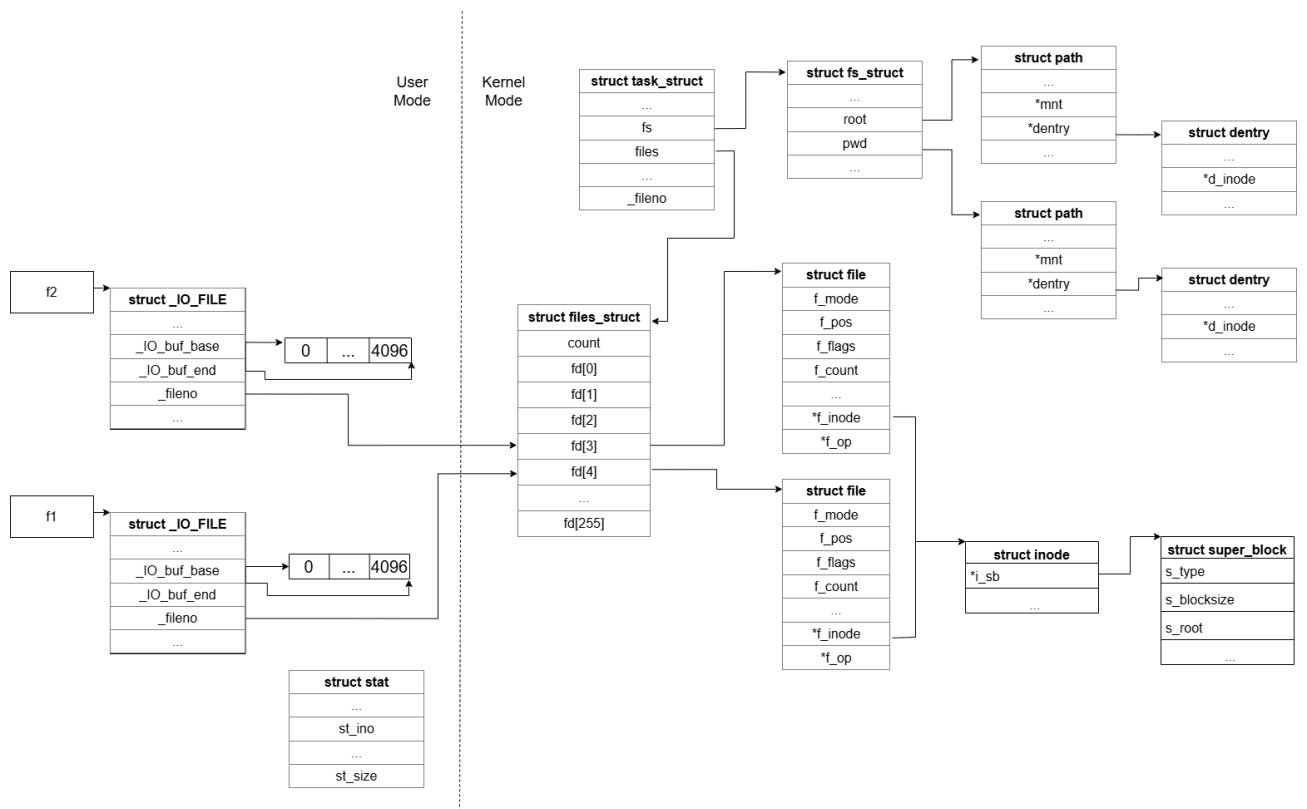



Рис. 2.9 — Связь структур в третьей программе

Буферизованный ввод. При вызове `open()` дважды открывается файл `3.txt`. Создаются два файловых дескриптора и две структуры `struct FILE`, поля `_fileno` которых инициализируются соответствующими значениями дескрипторов.

В цикле с помощью системного вызова (`write()`) производится запись чётных буквы алфавита в буфер первого дескриптора, нечётных — в буфер второго. Запись в файл из буфера происходит в одном из трёх случаев:

- буфер заполнен;
- вызвана функция `fflush()`;
- вызвана функция `fclose()`.

При вызове `fclose()` для первой структуры в файл из буфера записываются нечетные буквы латинского алфавита. Так как поле `f_pos` структуры `struct file`, соответствующей второму дескриптору, не изменилось, при последующем вызове `fclose()` чётные буквы записываются на те же позиции, перезаписывая ранее сохранённые данные. Вывод полей структуры `stat` подтверждает этот результат: размер файла не изменяется в ходе цикла, поскольку до закрытия дескриптора физической записи не происходит.

Небуферизованный ввод Системные вызовы `open()` открывают файл `3.txt` для чтения и записи, создавая дескрипторы со значениями 3 и 4 (значения 0, 1, 2 заняты стандартными потоками `stdin`, `stdout`, `stderr`), соответствующие индексам в таблице дескрипторов файлов процесса.

Поля `fd_array[3]` и `fd_array[4]` указывают на разные структуры `struct file`, связанные с одной и той же `struct inode`. Поскольку созданы две независимые структуры, их поля `f_pos` изменяются независимо. В результате чётные и нечётные буквы латинского алфавита записываются поочерёдно на одни и те же позиции файла, а размер файла не изменяется, что подтверждается выводом `stat()` в ходе выполнения программы.

Вывод

При работе с буферизированным вводом-выводом может возникнуть ряд проблем, которые демонстрируются в программах данной лабораторной работы. В первой программе использование `stdio`, который использует буферизованный ввод/вывод, приводит к тому, что данные сначала записываются в буфер, а затем выбираются из буфера. Во второй программе возникает проблема вывода символов на экран, так как из-за двух файловых дескрипторов смещения в файле будут производиться независимо. В третьей программе при одновременной записи данные могут быть перезаписаны, по этой причине необходимо открывать файл с опцией "a"(`O_APPEND`), тогда операция записи станет атомарной и данные не будут потеряны.